

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ  
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ**

**Факультет прикладной информатики**

**Образовательная программа Мобильные и сетевые технологии**

**Направление подготовки 09.03.03 Мобильные и сетевые технологии**

**О Т Ч Е Т**

Лабораторная работа №3

**Тема работы:** «Создание таблиц базы данных PostgreSQL. Заполнение таблиц рабочими данными»

**Обучающийся:** Федорова Мария Витальевна, К3239

**Поток:** ПиРБД К24 МСТ 1.2

**Преподаватель:** Говорова М.М.

Санкт-Петербург,  
2026

## 1. Цель работы

Овладеть практическими навыками создания таблиц базы данных PostgreSQL, заполнения их рабочими данными, резервного копирования и восстановления БД.

## 2. Практическое задание

В ходе выполнения лабораторной работы требовалось:

1. создать базу данных с использованием pgAdmin 4;
2. создать схему в составе базы данных;
3. создать таблицы базы данных;
4. установить ограничения на данные: Primary Key, Unique, Check, Foreign Key;
5. заполнить таблицы базы данных рабочими данными;
6. создать резервную копию базы данных;
7. восстановить базу данных.

## 3. Выполнение

### 3.1. Наименование БД и вариант

В ходе выполнения работы была реализована база данных `restaurant_service`.

Индивидуальный вариант: **Вариант 13. БД «Ресторан».**

### 3.2. Краткое описание предметной области

Необходимо создать систему для обслуживания заказов клиентов в ресторане. Сотрудники ресторана представлены официантами, поварами и шеф-поваром. Клиенты могут бронировать столы заранее. Официант принимает заказ от стола и передаёт его на кухню. Шеф-повар распределяет блюда между поварами. Один заказ может содержать несколько одинаковых или разных блюд. Для приготовления блюд используются ингредиенты, которые закупаются у поставщиков и хранятся на складе.

### 3.3. Схема инфологической модели БД в нотации IDEF1X

На рисунке 1 представлена инфологическая модель базы данных, разработанная в лабораторной работе №2.

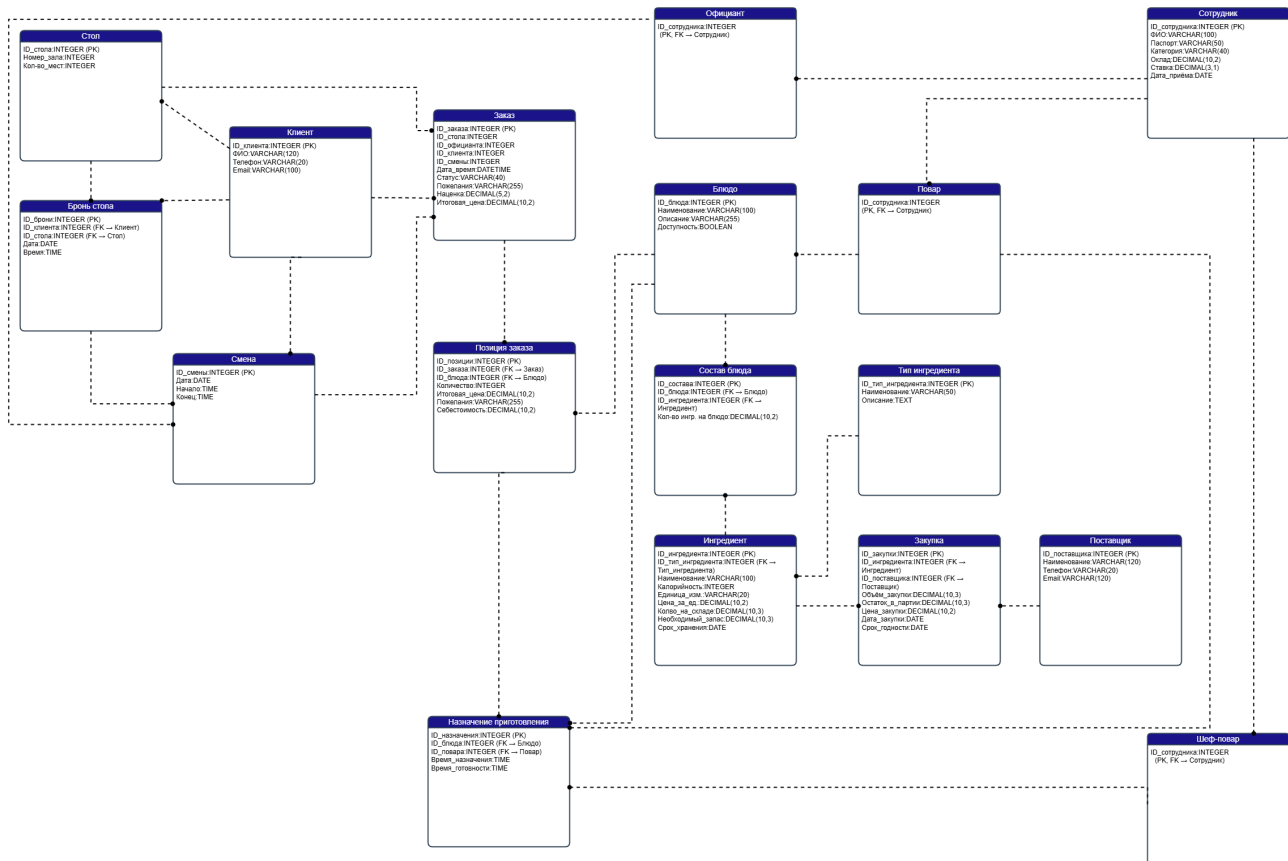


Рис. 1. Инфологическая модель БД в нотации IDEF1X

### 3.4. Схема логической модели базы данных, сгенерированная в Generate ERD

После создания таблиц, задания ограничений и связей в pgAdmin была сгенерирована логическая схема базы данных с помощью инструмента Generate ERD. Полученная диаграмма отражает структуру реализованной базы данных PostgreSQL.

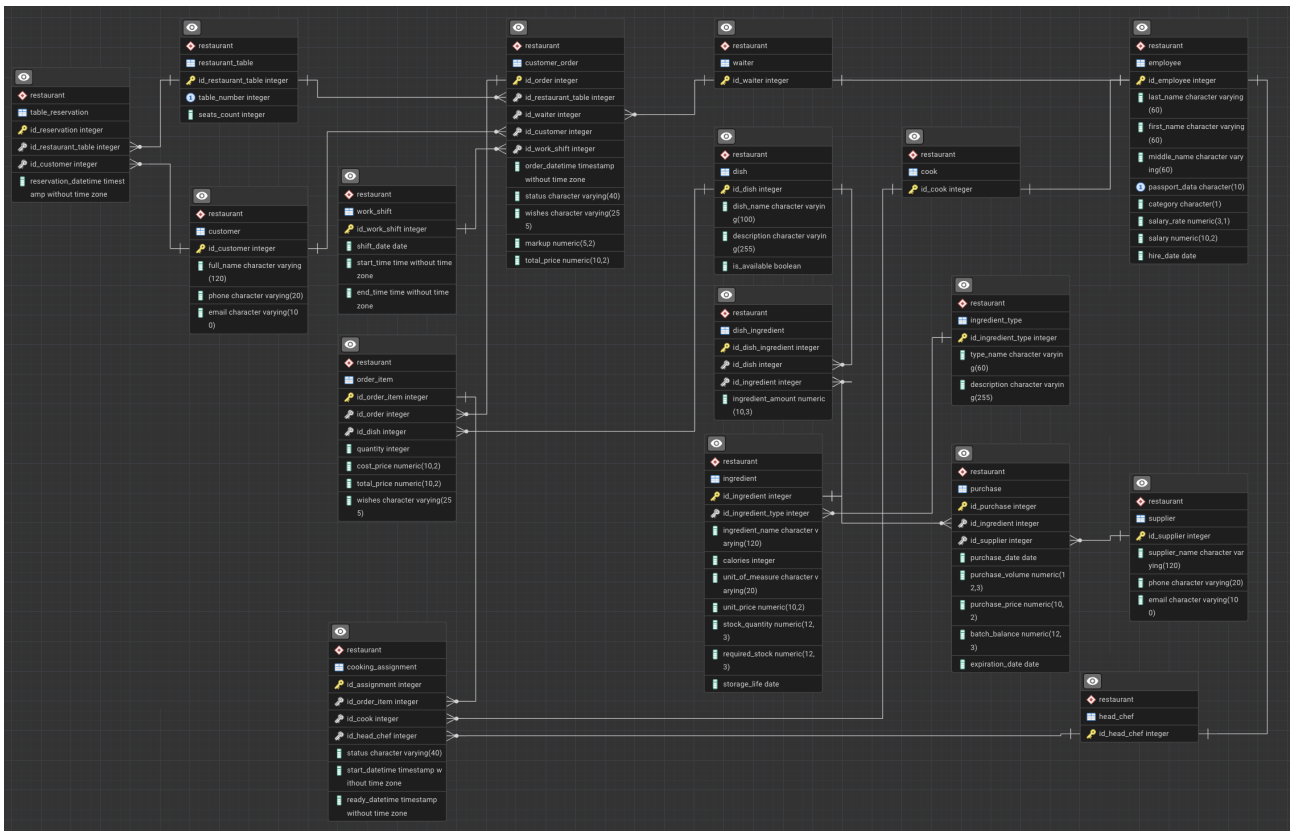


Рис. 2. Логическая модель базы данных, сгенерированная в Generate ERD

### 3.5. Анализ ключей и использование суррогатных идентификаторов

При переходе от инфологической модели к физической реализации базы данных были использованы суррогатные простые ключи таблиц. Для каждой таблицы был введён собственный идентификатор вида `id_...`, который выступает в роли первичного ключа. Такой подход упростил реализацию связей между таблицами и позволил избежать использования составных первичных ключей.

В результате были использованы следующие первичные ключи:

- `employee(id_employee);`
- `waiter(id_waiter);`
- `cook(id_cook);`
- `head_chef(id_head_chef);`
- `restaurant_table(id_restaurant_table);`
- `table_reservation(id_reservation);`
- `customer(id_customer);`
- `customer_order(id_order);`
- `order_item(id_order_item);`

- dish(id\_dish);
- dish\_ingredient(id\_dish\_ingredient);
- ingredient\_type(id\_ingredient\_type);
- ingredient(id\_ingredient);
- supplier(id\_supplier);
- purchase(id\_purchase);
- cooking\_assignment(id\_assignment);
- work\_shift(id\_work\_shift).

### 3.6. Создание схемы и таблиц

В базе данных была создана схема **restaurant**, внутри которой были размещены все таблицы предметной области. Были созданы следующие таблицы:

- employee;
- waiter;
- cook;
- head\_chef;
- restaurant\_table;
- table\_reservation;
- customer;
- customer\_order;
- order\_item;
- dish;
- dish\_ingredient;
- ingredient\_type;
- ingredient;
- supplier;
- purchase;
- cooking\_assignment;
- work\_shift.

### 3.7. Задание ограничений

Для обеспечения целостности данных в таблицах были заданы ограничения следующих типов:

- **Primary Key** — для идентификации записей в каждой таблице;
- **Foreign Key** — для задания ссылочной целостности между таблицами;
- **Unique** — для обеспечения уникальности отдельных атрибутов;
- **Check** — для контроля допустимых значений атрибутов.

Например:

- в таблице `employee` были заданы ограничения на допустимые значения категории сотрудника, ставки, оклада и даты приёма;
- в таблице `restaurant_table` были заданы ограничения на диапазон номера стола и количество мест;
- в таблицах `customer` и `supplier` были заданы проверки формата телефона и адреса электронной почты;
- в зависимых таблицах были заданы внешние ключи, обеспечивающие корректность связей между сотрудниками, заказами, блюдами, ингредиентами, поставщиками и закупками.

### 3.8. Заполнение таблиц рабочими данными

После создания структуры БД таблицы были заполнены рабочими данными с помощью инструмента Query Tool и команд `INSERT INTO`. Были добавлены тестовые записи для сотрудников, ролей сотрудников, столов, клиентов, смен, блюд, типов ингредиентов, ингредиентов, поставщиков, заказов, позиций заказа, состава блюд, закупок и назначений приготовления.

Для проверки корректности загрузки данных были выполнены запросы вида `SELECT * FROM <имя_таблицы>`.

### 3.9. Резервное копирование и восстановление БД

Для базы данных были созданы две резервные копии:

- резервная копия в формате `Custom` — для последующего восстановления;
- резервная копия в формате `Plain` — в виде текстового SQL-скрипта для включения в отчёт.

После создания резервных копий была выполнена процедура восстановления базы данных в отдельную БД `restaurant_service_restore`. Корректность восстановления была проверена выполнением запросов `SELECT` и сравнением результатов с исходной БД.

### 3.10. Dump со скриптами работы с БД

В соответствии с требованиями лабораторной работы в отчёт включён фрагмент dump-файла базы данных, содержащий команды создания базы данных, схемы, таблиц, определения ограничений и заполнения таблиц рабочими данными. Данный фрагмент отражает основные этапы физической реализации БД restaurant\_service в СУБД PostgreSQL.

Листинг 1. Фрагмент dump-файла базы данных restaurant\_service

```
-- Create database
CREATE DATABASE restaurant_service;

-- Connect to database
\connect restaurant_service

-- Create schema
CREATE SCHEMA restaurant;

-- Set search path
SET search_path TO restaurant;

-- Create employee table
CREATE TABLE employee (
    id_employee INTEGER NOT NULL,
    last_name CHARACTER VARYING(60) NOT NULL,
    first_name CHARACTER VARYING(60) NOT NULL,
    middle_name CHARACTER VARYING(60),
    passport_data CHARACTER(10) NOT NULL,
    category CHARACTER(1) NOT NULL,
    salary_rate NUMERIC(3,1) NOT NULL,
    salary NUMERIC(10,2) NOT NULL,
    hire_date DATE NOT NULL
);

-- Employee constraints
ALTER TABLE employee
    ADD CONSTRAINT pk_employee PRIMARY KEY (id_employee);

ALTER TABLE employee
    ADD CONSTRAINT uq_employee_passport_data UNIQUE (passport_data);

ALTER TABLE employee
    ADD CONSTRAINT chk_employee_category
    CHECK (category IN ('A', 'B', 'C', 'D'));

ALTER TABLE employee
    ADD CONSTRAINT chk_employee_salary_rate
    CHECK (salary_rate >= 0.1 AND salary_rate <= 1.0);

ALTER TABLE employee
    ADD CONSTRAINT chk_employee_salary
    CHECK (salary > 0);

ALTER TABLE employee
```

```

    ADD CONSTRAINT chk_employee_hire_date
    CHECK (hire_date <= CURRENT_DATE);

-- Create customer table
CREATE TABLE customer (
    id_customer INTEGER NOT NULL,
    full_name CHARACTER VARYING(120) NOT NULL,
    phone CHARACTER VARYING(20) NOT NULL,
    email CHARACTER VARYING(100)
);

-- Customer constraints
ALTER TABLE customer
    ADD CONSTRAINT pk_customer PRIMARY KEY (id_customer);

ALTER TABLE customer
    ADD CONSTRAINT chk_customer_phone
    CHECK (phone ~ '^+7[0-9]{10}$');

ALTER TABLE customer
    ADD CONSTRAINT chk_customer_email
    CHECK (
        email IS NULL OR
        email ~ '^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})$'
    );

-- Create restaurant table
CREATE TABLE restaurant_table (
    id_restaurant_table INTEGER NOT NULL,
    table_number INTEGER NOT NULL,
    seats_count INTEGER NOT NULL
);

-- Restaurant table constraints
ALTER TABLE restaurant_table
    ADD CONSTRAINT pk_restaurant_table PRIMARY KEY (id_restaurant_table);

ALTER TABLE restaurant_table
    ADD CONSTRAINT uq_restaurant_table_table_number UNIQUE (table_number);

ALTER TABLE restaurant_table
    ADD CONSTRAINT chk_restaurant_table_number
    CHECK (table_number >= 1 AND table_number <= 40);

ALTER TABLE restaurant_table
    ADD CONSTRAINT chk_restaurant_table_seats_count
    CHECK (seats_count > 0);

-- Create waiter table
CREATE TABLE waiter (
    id_waiter INTEGER NOT NULL
);

```



```

ALTER TABLE waiter
    ADD CONSTRAINT pk_waiter PRIMARY KEY (id_waiter);

ALTER TABLE waiter
    ADD CONSTRAINT fk_waiter_employee
    FOREIGN KEY (id_waiter) REFERENCES employee(id_employee);

-- Create work shift table
CREATE TABLE work_shift (
    id_work_shift INTEGER NOT NULL,
    shift_date DATE NOT NULL,
    start_time TIME WITHOUT TIME ZONE NOT NULL,
    end_time TIME WITHOUT TIME ZONE NOT NULL
);

ALTER TABLE work_shift
    ADD CONSTRAINT pk_work_shift PRIMARY KEY (id_work_shift);

ALTER TABLE work_shift
    ADD CONSTRAINT chk_work_shift_time
    CHECK (end_time > start_time);

-- Create customer order table
CREATE TABLE customer_order (
    id_order INTEGER NOT NULL,
    id_restaurant_table INTEGER NOT NULL,
    id_waiter INTEGER NOT NULL,
    id_customer INTEGER NOT NULL,
    id_work_shift INTEGER NOT NULL,
    order_datetime TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    status CHARACTER VARYING(40) NOT NULL,
    wishes CHARACTER VARYING(255),
    markup NUMERIC(5,2) NOT NULL,
    total_price NUMERIC(10,2) NOT NULL
);

-- Customer order constraints
ALTER TABLE customer_order
    ADD CONSTRAINT pk_customer_order PRIMARY KEY (id_order);

ALTER TABLE customer_order
    ADD CONSTRAINT fk_customer_order_restaurant_table
    FOREIGN KEY (id_restaurant_table)
    REFERENCES restaurant_table(id_restaurant_table);

ALTER TABLE customer_order
    ADD CONSTRAINT fk_customer_order_waiter
    FOREIGN KEY (id_waiter)
    REFERENCES waiter(id_waiter);

ALTER TABLE customer_order
    ADD CONSTRAINT fk_customer_order_customer
    FOREIGN KEY (id_customer)

```

```

REFERENCES customer(id_customer);

ALTER TABLE customer_order
  ADD CONSTRAINT fk_customer_order_work_shift
  FOREIGN KEY (id_work_shift)
  REFERENCES work_shift(id_work_shift);

ALTER TABLE customer_order
  ADD CONSTRAINT chk_customer_order_markup
  CHECK (markup >= 0);

ALTER TABLE customer_order
  ADD CONSTRAINT chk_customer_order_total_price
  CHECK (total_price >= 0);

-- Create ingredient type table
CREATE TABLE ingredient_type (
  id_ingredient_type INTEGER NOT NULL,
  type_name CHARACTER VARYING(60) NOT NULL,
  description CHARACTER VARYING(255)
);

ALTER TABLE ingredient_type
  ADD CONSTRAINT pk_ingredient_type PRIMARY KEY (id_ingredient_type);

-- Create ingredient table
CREATE TABLE ingredient (
  id_ingredient INTEGER NOT NULL,
  id_ingredient_type INTEGER NOT NULL,
  ingredient_name CHARACTER VARYING(120) NOT NULL,
  calories INTEGER NOT NULL,
  unit_of_measure CHARACTER VARYING(20) NOT NULL,
  unit_price NUMERIC(10,2) NOT NULL,
  stock_quantity NUMERIC(12,3) NOT NULL,
  required_stock NUMERIC(12,3) NOT NULL,
  storage_life DATE NOT NULL
);

ALTER TABLE ingredient
  ADD CONSTRAINT pk_ingredient PRIMARY KEY (id_ingredient);

ALTER TABLE ingredient
  ADD CONSTRAINT fk_ingredient_ingredient_type
  FOREIGN KEY (id_ingredient_type)
  REFERENCES ingredient_type(id_ingredient_type);

ALTER TABLE ingredient
  ADD CONSTRAINT chk_ingredient_calories
  CHECK (calories >= 0);

ALTER TABLE ingredient
  ADD CONSTRAINT chk_ingredient_unit_price
  CHECK (unit_price > 0);

```

```

ALTER TABLE ingredient
  ADD CONSTRAINT chk_ingredient_stock_quantity
  CHECK (stock_quantity >= 0);

ALTER TABLE ingredient
  ADD CONSTRAINT chk_ingredient_required_stock
  CHECK (required_stock >= 0);

-- Insert data into employee
INSERT INTO employee (
  id_employee, last_name, first_name, middle_name,
  passport_data, category, salary_rate, salary, hire_date
) VALUES
(1, 'Ivanov', 'Ivan', 'Ivanovich', '1234567890', 'A', 1.0, 90000.00, '2024-01-10'),
(2, 'Petrova', 'Anna', 'Sergeevna', '2345678901', 'B', 1.0, 85000.00, '2024-02-15'),
(3, 'Sidorov', 'Pavel', 'Olegovich', '3456789012', 'B', 1.0, 80000.00, '2024-03-05'),
(4, 'Smirnova', 'Elena', 'Igorevna', '4567890123', 'C', 0.5, 45000.00, '2024-04-01');

-- Insert data into waiter
INSERT INTO waiter (id_waiter) VALUES
(1);

-- Insert data into customer
INSERT INTO customer (
  id_customer, full_name, phone, email
) VALUES
(1, 'Fedorova Maria', '+79991234567', 'maria@example.com'),
(2, 'Pakhomov Gleb', '+79997654321', 'gleb@example.com'),
(3, 'Ivanova Olga', '+79990001122', NULL);

-- Insert data into restaurant_table
INSERT INTO restaurant_table (
  id_restaurant_table, table_number, seats_count
) VALUES
(1, 1, 2),
(2, 2, 4),
(3, 3, 6);

-- Insert data into work_shift
INSERT INTO work_shift (
  id_work_shift, shift_date, start_time, end_time
) VALUES
(1, '2026-03-20', '09:00:00', '17:00:00'),
(2, '2026-03-20', '17:00:00', '23:00:00');

-- Insert data into ingredient_type
INSERT INTO ingredient_type (
  id_ingredient_type, type_name, description
) VALUES
(1, 'Vegetable', 'Fresh vegetables'),
(2, 'Meat', 'Meat products'),
(3, 'Dairy', 'Milk products'),

```

```

(4, 'Spice', 'Seasonings');

-- Insert data into ingredient
INSERT INTO ingredient (
    id_ingredient, id_ingredient_type, ingredient_name, calories,
    unit_of_measure, unit_price, stock_quantity, required_stock, storage_life
) VALUES
(1, 1, 'Tomato', 18, 'kg', 120.00, 15.000, 5.000, '2026-03-30'),
(2, 2, 'Chicken', 239, 'kg', 420.00, 10.000, 4.000, '2026-03-25'),
(3, 3, 'Parmesan', 431, 'kg', 900.00, 3.000, 1.000, '2026-04-10'),
(4, 4, 'Black Pepper', 251, 'kg', 700.00, 1.000, 0.200, '2026-12-31');

-- Insert data into customer_order
INSERT INTO customer_order (
    id_order, id_restaurant_table, id_waiter, id_customer, id_work_shift,
    order_datetime, status, wishes, markup, total_price
) VALUES
(1, 2, 1, 1, 2, '2026-03-20 18:35:00', '', 'No onion', 40.00, 980.00),
(2, 1, 1, 2, 2, '2026-03-20 19:05:00', '', NULL, 35.00, 735.00);

```

#### 4. Выводы

В ходе выполнения лабораторной работы была реализована база данных **restaurant\_service** в СУБД PostgreSQL с использованием pgAdmin 4. Была создана схема **restaurant**, сформирована структура таблиц, заданы ограничения целостности, загружены рабочие данные, выполнено резервное копирование и проверено восстановление БД.

В результате были приобретены практические навыки:

- создания базы данных и схемы в pgAdmin 4;
- проектирования и реализации таблиц PostgreSQL;
- задания ограничений Primary Key, Unique, Check и Foreign Key;
- заполнения таблиц рабочими данными с помощью Query Tool;
- создания резервных копий в форматах Custom и Plain;
- восстановления базы данных из резервной копии.

Таким образом, цель лабораторной работы достигнута: получены практические навыки создания таблиц базы данных PostgreSQL, заполнения их рабочими данными, резервного копирования и восстановления базы данных.